

→ How Machine Learning Works or What are the stages?

1) Decide on the Problem → What are the inputs and outputs (What are we modeling)

2) Curate Data → get the data, cleaning, preparing the data (What data will inform the model)

3) Design an Architecture → Choose an architecture that will represent (RNN, Autoencoder, DMD, SINDy) that function relationship.

4) crafting a loss function → the actual error of the input-output prediction of your NN or architecture (What models are 'good'?)

5) Employ Optimisation → We use optimisation to tweak the parameters of our architecture to learn the parameter to minimize the loss function averaged over our training data. (What algorithms to train model?)

Each of those five stages gives unique opportunity for Embedding Physics into the process, and in some cases for discovering Physics

→ Our Architecture is some class of functions that you can use to represent the input output relationships you want to model.

→ They are parameters of this architecture that we can tune. ⇒ In NN there are the weights.

Some times you can
Some times you can't

From Alchemy

→ a critical metaphor used to describe the practice of building models through empirical trial and error without a complete theoretical understanding of why they work.

→ There are lot of other Architectures that are not NN that we can use to learn models from data

→ We have some idea of NN that may work or may not for our problem. So we try bunch of them → some work, some not

→ So one of the big pushes of applying machine learning to a physical systems where we actually do have some prior knowledge, is that maybe we can also start learning principles (we know a lot about these systems) of how and when different NN architectures actually are appropriate for a given problem

★ [We can learn about these algorithms by applying them where we already know the answer then to apply where we don't know the answer.]

We can go from Alchemy to like chemistry [where there are actual principles and organization to when and how to use those building blocks for particular problem.]

Example: like we know about elements and periodic table
↳ we can organize these architectures like elements in periodic table

So we apply ML in physics where we already know the answer.

Look up
AI Institute in
dynamic system

Stage 1 → Decide on the Problem → modeling a physical system

Stage 1 → Decide on the Problem → modeling a physical system

- ↳ When you are trying to model a physical process, physics is already in the process.
- eg:- modeling a pendulum, lava lamp, fluids flow
- ↳ knowing the forces of pendulum
- ↳ setting up the problem in such a way that it includes physics.

Stage 2 → Curate the data → Data comes from physical system.

- ↳ eg: going to take fluid flow fields as my training data
- ↳ then its going to have them at different flow velocities or has a range of velocities.
- eg: geophysical fluid flow data → cloud formation data

↳ You can take a data and augment it to create variants of the data

⇒ Using data like coordinate of solar system for model (includes physics)

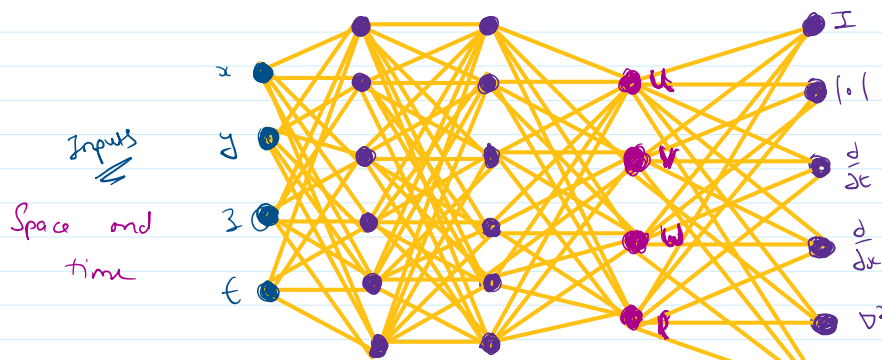
Stage 3 → Design an Architecture → Using Architecture like Lagrangian NN for a pendulum model
→ Model like these have their own loss functions

Stage 4 → CRAFT a loss function

→ Physics is promoted through additional regularization terms or loss functions in your training process.

eg: Physics Informed NN or PINN

→ What the PINN does



→ Normal Model $\sum \|Q(x_i) - u(x_i)\|_2^2$

→ We apply loss function to check how good the model is

→ But its a naive way of doing

Virtual $\sum \|N(u(x_i), x_i, t)\|_2^2$

- ↳ From output data you can compute terms in partial differential equation
- ↳ And you can add actual physics the partial differential equation that the system should be satisfying as another loss term in your loss function

↳ If you know the physics or if you know the system is divergence free or satisfy the Navier Stokes equations or the elastic beam equation of whatever equation

↳ If you know it, you can add in a regularizing loss term to your ML process.

↳ This should dramatically improve the performance of model

⇒ Much better model performance if you add physics as loss function

Stage 5 → Employ Optimization → Gives more physics But more human effort to add that optimization.
⇒ Using optimization to enforce physics

↳ Last stage we did → added loss function → That was basically a suggestion that we want physics to be satisfied.

last stage we did $\rightarrow$ added loss function $\rightarrow$ that was basically a suggestion that we want physics to be satisfied.
 $\hookrightarrow$ Adding physics as a loss function only kind of promotes physics but it doesn't enforce that physics is satisfied

$\Rightarrow$ But with optimization you can constrain and enforce that the physics is satisfied at a much higher precision.

$\Rightarrow$ If I know some constraints that should be satisfied for my system to be physical things like energy conservation or certain type of symmetry, I can either add that in as term in my loss function

OR we can do things like use lrange multipliers or constrained least squares so that I can change my optimization problem so that those constraints are exactly satisfied, every time I optimize this function

Example of those 5 stages = Problem is to take a complicated model of a fluid and I want a simpler model of that fluid

1. Problem (eg: model reduction)
 $\downarrow$
 Setting up the Problem

• Something that runs faster on a smaller computer

2. Data X

Data $\hookrightarrow$ Numerical simulation of a fluid flow

• Start with a high dimensional data

• Choose an architecture

• In this case its a kind of autoencoder architecture because it has the properties we want

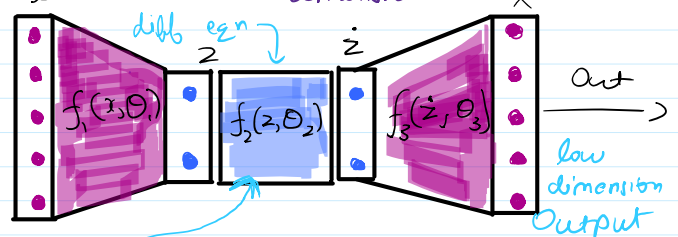
$\hookrightarrow$ It goes from high dimension to low dimension

$\Rightarrow$ In this case a simulation an expensive simulation of a full complexity

3. Architecture $f(x, \theta) = f_3 \circ f_2 \circ f_1$

high dimension input

Data In



• Take a high dimensional system, compress it down to a low dimensional state and learn some dynamics, some differential equation in that low dimensional state

• These low dimension states z are chosen so that they are most likely to be able to reconstruct the high dimensional state again.

• It will be much easier and faster to run this simulation maybe in real time and we will get most of the physics that's happening in this system.

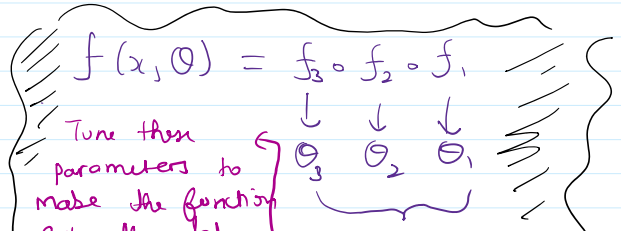
• We also want to be able to learn a dynamical system some differential equation on that low dimensional latent state
 • That's what mean by model reduction

4) Crafting a loss function $\mathcal{L}(\theta, x)$

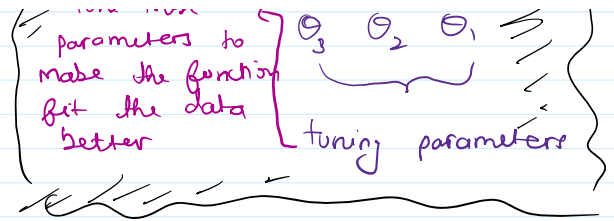
• Our architecture is some function that maps input to output, to solve the problem

• It have parameter Θ

(these parameters are learnable)



- It has parameter Θ
- ↳ these parameters are tuning parameters, that we can tweak to make this function fit this data better



5) Optimize $\Theta^* = \operatorname{argmin}_{\Theta} \mathcal{L}(\Theta, X)$

- Optimize over all of these Θ s, to minimize the loss function averaged over the training data
- This is the restatement of all we have done